Reporte de Práctica: Laboratorio 02

INTEGRANTES:

CASTILLO HERNANDEZ REBECA

CARREÓN HERNANDEZ EMMANUEL

MEDELLÍN LÓPEZ MARTIN JOSAFATH

RODRIGUEZ OSORIO DIEGO SALAZAR

LARES AYLIN XIMENA

8-J

DISEÑO ELECTRONICO BASADO EN SISTEMAS EMBEBIDOS.

DOCENTE: DANIEL LÓPEZ PIÑA.

ABRIL, 2026

Reporte de Práctica: Laboratorio 02

Materia: Diseño Electrónico Basado en Sistemas Embebidos

Tema: Entradas Digitales — Pulsadores y Pull-up/Pull-down

1. Objetivos

Configurar pines GPIO como entradas digitales utilizando resistencias Pull-up internas.

Implementar una solución de debounce (anti-rebote) por software de forma no bloqueante (usando millis()).

Controlar el flujo del programa mediante eventos de entrada (pulsadores) y retroalimentación visual (LEDs).

Utilizar el Monitor Serial para el conteo de eventos y depuración de estados.

2. Fundamentos Teóricos

En esta práctica se trabajó con la configuración INPUT_PULLUP, la cual utiliza una resistencia interna de aproximadamente 50 kΩ. Esta configuración es fundamental porque:

Evita estados flotantes: Garantiza un nivel HIGH constante cuando el botón no está presionado.

Lógica inversa: Al presionar el botón, el pin se conecta a GND, leyendo un estado LOW.

Debounce: Se filtraron los rebotes mecánicos (~5–20 ms) para asegurar que cada pulsación física se registre como un único evento digital.

3. Desarrollo de Ejercicios

Ejercicio 1: Contador con Botón de Reset (Pin 2 y Pin 3)

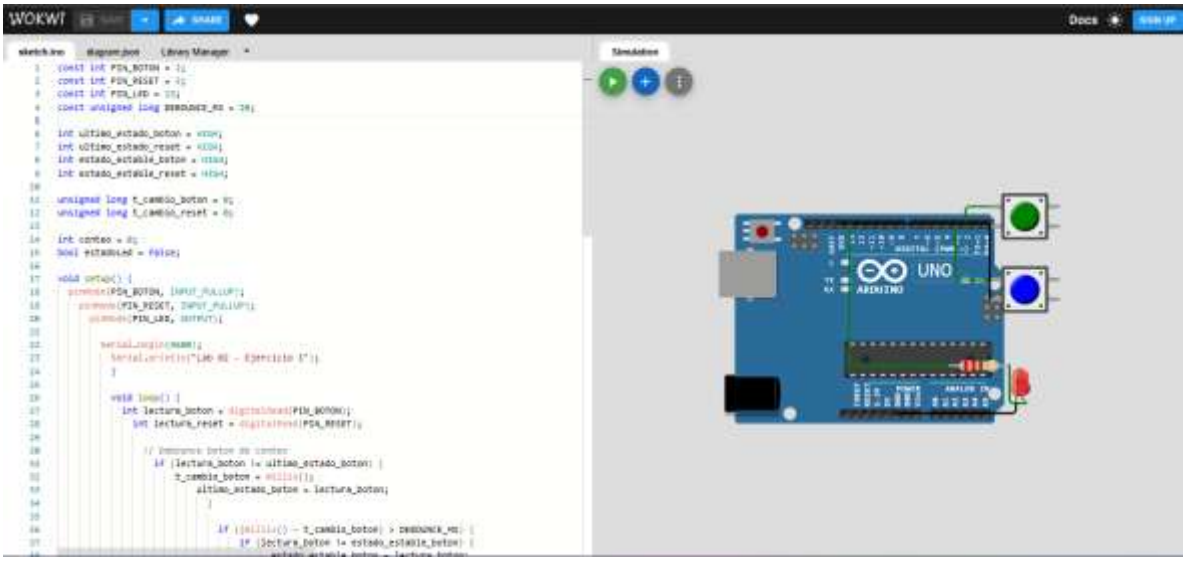
Se implementó un sistema con dos entradas:

Botón en Pin 2: incrementa el contador y alterna el estado del LED (toggle).

Botón en Pin 3: reinicia las variables.

Código: (El que proporcionaste con t_cambio_boton y t_cambio_reset).

Logro: Se logró un control independiente de cada botón aplicando debounce individual.



Ejercicio 1:

```
const int PIN_BOTON = 2;

const int PIN_RESET = 3;

const int PIN_LED = 13;

const unsigned long DEBOUNCE_MS = 20;


int ultimo_estado_boton = HIGH;

int ultimo_estado_reset = HIGH;

int estado_estable_boton = HIGH;

int estado_estable_reset = HIGH;


unsigned long t_cambio_boton = 0;

unsigned long t_cambio_reset = 0;


int conteo = 0;

bool estadoLed = false;
```

```
void setup() {
  pinMode(PIN_BOTON, INPUT_PULLUP);
    pinMode(PIN_RESET, INPUT_PULLUP);
      pinMode(PIN_LED, OUTPUT);

        Serial.begin(9600);
          Serial.println("Lab 02 - Ejercicio 1");
          }

          void loop() {
            int lectura_boton = digitalRead(PIN_BOTON);
              int lectura_reset = digitalRead(PIN_RESET);

                // Debounce boton de conteo
                  if (lectura_boton != ultimo_estado_boton) {
                     t_cambio_boton = millis();
                        ultimo_estado_boton = lectura_boton;
                          }

                            if ((millis() - t_cambio_boton) > DEBOUNCE_MS) {
                               if (lectura_boton != estado_estable_boton) {
                                    estado_estable_boton = lectura_boton;

                                         if (estado_estable_boton == LOW) {
                                              conteo++;
                                                   estadoLed = !estadoLed;
                                                        digitalWrite(PIN_LED, estadoLed);

                                                             Serial.print("Pulsacion #");
```

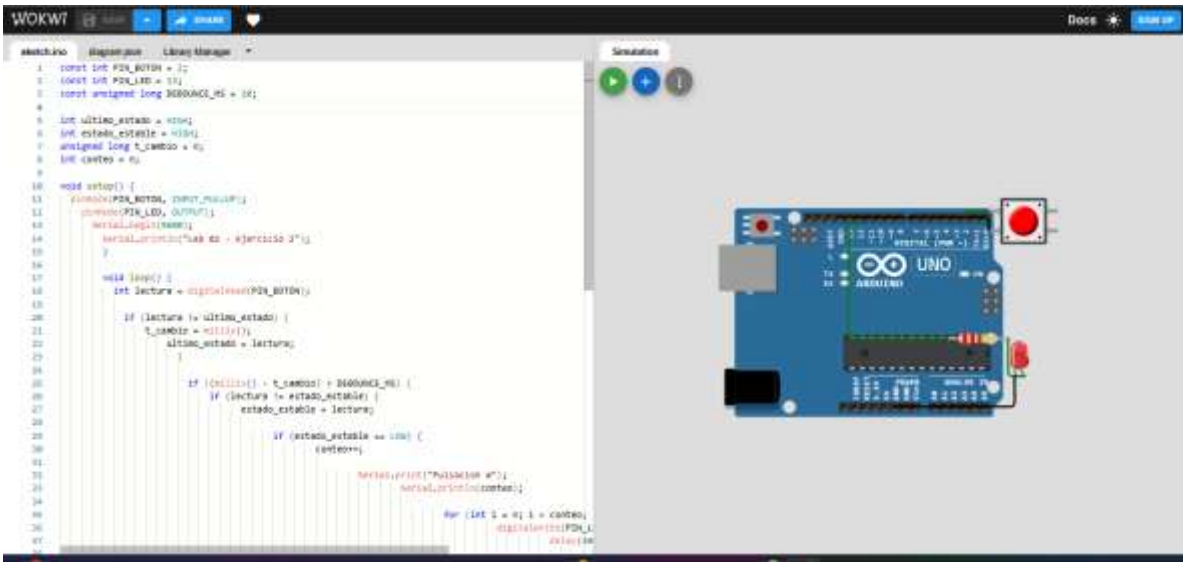
```
Serial.println(conteo);
    }
      }
       }

        // Debounce boton de reset
         if (lectura_reset != ultimo_estado_reset) {
           t_cambio_reset = millis();
             ultimo_estado_reset = lectura_reset;
              }

               if ((millis() - t_cambio_reset) >
DEBOUNCE_MS) {
                 if (lectura_reset !=
estado_estable_reset) {
                   estado_estable_reset =
lectura_reset;

                     if (estado_estable_reset
== LOW) {
                       conteo = 0;
                         estadoLed =
false;
digitalWrite(PIN_LED, LOW);
Serial.println("CONTEO REINICIADO");
                                    }
                                      }
                                       }
                                       }
```

Ejercicio 2: Retroalimentación Visual con Bucle for

En este ejercicio, cada vez que se valida una pulsación, el sistema ejecuta una rutina de parpadeo.

Funcionamiento: Si el conteo es 3, el ciclo for (int i = 0; i < conteo; i++) hace que el LED parpadee exactamente 3 veces con intervalos de 300 ms.

Análisis: Se observó que mientras el LED parpadea, el programa no detecta nuevas pulsaciones debido al uso de delay() dentro del bucle.



Ejercicio 2:

```
const int PIN_BOTON = 2;

const int PIN_LED = 13;

const unsigned long DEBOUNCE_MS = 20;


int ultimo_estado = HIGH;

int estado_estable = HIGH;

unsigned long t_cambio = 0;

int conteo = 0;


void setup() {
```

```
pinMode(PIN_BOTON, INPUT_PULLUP);
  pinMode(PIN_LED, OUTPUT);
    Serial.begin(9600);
      Serial.println("Lab 02 - Ejercicio 2");
      }

      void loop() {
        int lectura = digitalRead(PIN_BOTON);

          if (lectura != ultimo_estado) {
              t_cambio = millis();
                ultimo_estado = lectura;
                  }

                    if ((millis() - t_cambio) > DEBOUNCE_MS) {
                        if (lectura != estado_estable) {
                            estado_estable = lectura;

                                if (estado_estable == LOW) {
                                      conteo++;

                                            Serial.print("Pulsacion #");
                                                  Serial.println(conteo);

                                                        for (int i = 0; i < conteo; i++) {
                                                              digitalWrite(PIN_LED, HIGH);
                                                                    delay(300);
                                                                          digitalWrite(PIN_LED, LOW);
                                                                                delay(300);
```

}

}

}

}

}

Ejercicio 3: Control Dual (LED Verde y Rojo)

Se simuló un sistema de control directo.

Configuración:

Botón A → LED Verde (Pin 8)

Botón B → LED Rojo (Pin 9)

Resultado: Se validó la lectura simultánea de múltiples entradas digitales.

4. Lista de Materiales (BOM - Lab 02)

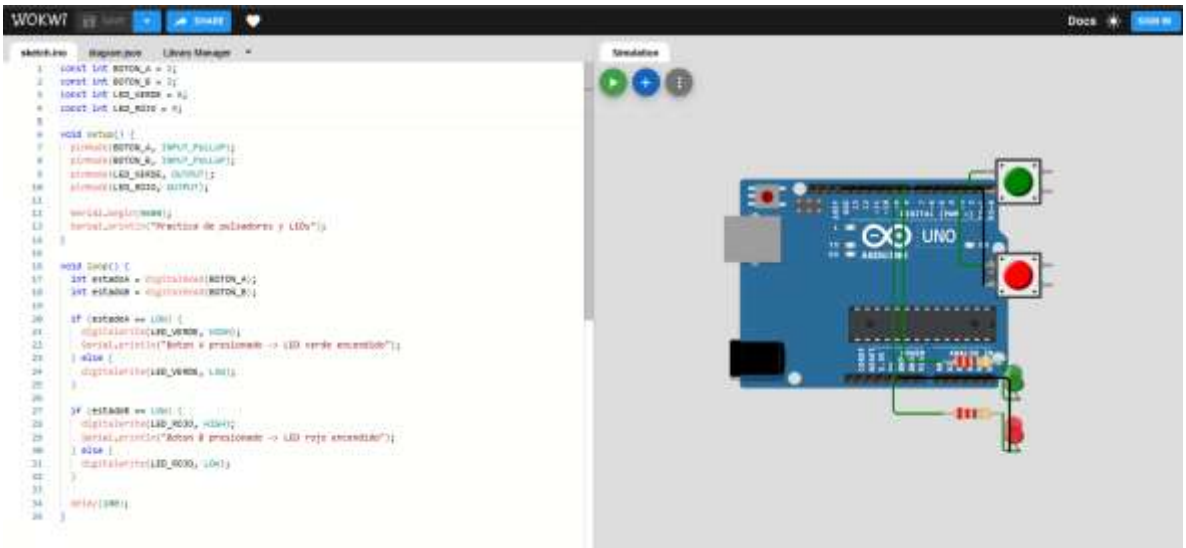
Cantidad	Componente	Descripción
1	Arduino UNO	Microcontrolador principal
2	Pulsadores	Interruptores momentáneos de 4 pines
1	LED Verde	Indicador de estado (Pin 8 o 13)
1	LED Rojo	Indicador de estado (Pin 9)
2	Resistencias 220 Ω	Para protección de los LEDs
n	Jumpers	Cables para conexión en protoboard

5. Resultados y Conclusiones

Debounce no bloqueante: Se demostró que el uso de millis() es superior al delay() para el debounce, ya que permite que el loop() siga ejecutándose continuamente.

Pull-up interno: Se simplificó el circuito físico al no requerir resistencias externas de 10 kΩ, utilizando la función pinMode(PIN, INPUT_PULLUP).

Manejo de flancos: Se comprendió que, para detectar una pulsación con Pull-up, se debe buscar el flanco de bajada (cuando el estado pasa de HIGH a LOW).



Ejercicio 3:

```
const int BOTON_A = 2;

const int BOTON_B = 3;

const int LED_VERDE = 8;

const int LED_ROJO = 9;


void setup() {

  pinMode(BOTON_A, INPUT_PULLUP);

  pinMode(BOTON_B, INPUT_PULLUP);

  pinMode(LED_VERDE, OUTPUT);

  pinMode(LED_ROJO, OUTPUT);


  Serial.begin(9600);

  Serial.println("Practica de pulsadores y LEDs");

}


void loop() {

  int estadoA = digitalRead(BOTON_A);

  int estadoB = digitalRead(BOTON_B);
```

```
  if (estadoA == LOW) {
    digitalWrite(LED_VERDE, HIGH);
    Serial.println("Boton A presionado -> LED verde encendido");
  } else {
    digitalWrite(LED_VERDE, LOW);
  }

  if (estadoB == LOW) {
    digitalWrite(LED_ROJO, HIGH);
    Serial.println("Boton B presionado -> LED rojo encendido");
  } else {
    digitalWrite(LED_ROJO, LOW);
  }

  delay(100);
}
```